

CS 61C

Summer 2024

Andrew, Eddy, Jedi, Nikhil

Midterm

Solutions last updated: Monday, July 15th, 2024

PRINT Your Name: _____

PRINT Your Student ID: _____

You have 110 minutes. There are 8 questions of varying credit (100 points total).

Question:	1	2	3	4	5	6	7	8	Total
Points:	20	21	13	22	9	7	8	0	100

For questions with **circular bubbles**,
you may select only one choice.

- ☐ Unselected option (completely unfilled)
- ☒ Only one selected option (completely filled)
- ☒ Don't do this (it will be graded as incorrect)

For questions with **square checkboxes**,
you may select one or more choices.

- ☐ You can select
- ☒ multiple squares
- ☒ (completely filled)

Anything you write outside the answer boxes or you ~~cross-out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

If an answer requires hex input, you must only use capitalized letters (**0xDEADBEEF** instead of **0xdeadbeef**). For hex and binary, please include prefixes in your answers unless otherwise specified, and do not truncate any leading 0's. For all other bases, do not add any prefixes or suffixes.

Write the statement below in the same handwriting you will use on the rest of the exam.

I have neither given nor received help on this exam (or quiz), and have rejected any attempt to cheat; if these answers are not my own work, I may be deducted up to 0x0123 4567 89AB CDEF points.

SIGN your name: _____

Clarifications made during the exam:

Q1.13: This question will be dropped. Every student will receive full points for this question.

Q2.13: **friends** refers to the argument of **init**.

Q1 Potpourri

(20 points)

For Q1.1 – Q1.2, convert the 1-byte hexadecimal 0x92 to decimal, treating it as...

Q1.1 (1.5 points) ...an **unsigned** integer.

Solution: 146

Q1.2 (1.5 points) ...a **two's complement** integer.

Solution: –110

Q1.3 (1 point) Attack of the flippy bit! Convert the 1-byte hexadecimal 0xBD to binary.

Solution: 0b1011 1101.

Q1.4 (2 points) On a computer that only supports 6-bit, two's complement arithmetic, what is the result of 0x2C + 0x1D, in hexadecimal?

Solution: 0x09. The full answer would be 0x49, but since we have only 6-bit numbers, the MSB is truncated, leaving us with our answer.

Q1.5 (1 point) Which of the following represents the exact range $[-2^{n-1} + 1, 2^{n-1} - 1]$, where n is the number of bits used?

- ☐ Bias notation
- ☒ Sign and magnitude
- ☐ Two's complement
- ☐ Unsigned

Q1.6 (2 points) Consider the following **struct** definition, assuming that all relevant standard libraries have been imported.

```
1 struct minion {  
2     char bob;  
3     char *stuart;  
4     uint32_t jerry;  
5     char otto;  
6 };
```

On a 32-bit system, what is `sizeof(struct minion)`?

Solution: 16 bytes. `bob` is padded out to a 4 byte boundary as `sizeof minion.stuart == 4`, and `otto` is padded out to a 4 byte boundary since the size of the entire `minion` struct must be a multiple of its largest member.

For Q1.7 – Q1.9, refer to the following C code executed on a 32-bit, little-endian system. Assume all relevant standard libraries are imported and the code compiles and runs without errors.

```
1 // arr is located at the address 0x1000 3058
2 uint32_t arr[3] = {31, 41, 592};
3 printf("%p", arr + 1);
4 printf("%u", *(arr + 1));
5 printf("%d", strlen((char*) &arr[2]));
```

Q1.7 (1.5 points) What does line 3 print?

Solution: 0x1000 305C.

Q1.8 (1.5 points) What does line 4 print?

Solution: 41.

Q1.9 (1.5 points) What does line 5 print?

Solution: 2. The third number in `arr` converted to hexadecimal is `0x250`. Stored in a little endian system, we would have the bytes in the following order: `0x5002 0000`, meaning that `strlen` would return 2.

Q1.10 (1.5 points) Which of these are found in the output of the linker?

- ☒ Header
- ☐ Footer
- ☒ Text segment
- ☐ None of the above
- ☒ Data segment

Q1.11 (1.5 points) Which stage of CALL may contain pseudoinstructions in its output?

- ☒ Compiler
- ☐ Assembler
- ☐ Linker
- ☐ Loader

Q1.12 (2 points) Given the following RISC-V code, translate the instruction on line 4 into hexadecimal.

```
1 Label: li a3 0xDEADBEEF
2      andi a3 a3 100
3      or s0 s0 a3
4      bge a3 s0 Label
```

Solution: `0xFE86D8E3`. Since the immediate in the `li` pseudoinstruction is greater than 12 bits, `li` expands into two instructions, so the offset is `-16`.

(Question 1 continued...)

Q1.13 (1.5 points) Given an FSM with n states and two transitions from each state, how many distinct length- m sequences of transitions are there? Express your answer in terms of n and m .

Solution: This question was dropped from the exam. All students received full credit for this question.

Q2 61Companionship

(21 points)

The `profile_t` struct is defined as follows:

```
1 typedef struct profile {
2     uint32_t id;
3     char *addr;
4     uint32_t *friends;
5 } profile_t;
```

The function `friendlen` is defined as follows:

- Arguments:
 - `uint32_t *friends`: a zero-terminated array of `ids`
- Returns a `size_t`: the number of `ids` in `friends`, excluding the zero-terminator

The function `init` is defined as follows:

- Arguments:
 - `uint32_t id`: the `id` of the new `profile_t`
 - `char *(*addr_fn)(uint32_t)`: a pointer to a function that returns a string on the heap, which is the `addr` associated with the argument of this function
 - `uint32_t *friends`: a zero-terminated array of `uint32_ts` containing the `ids` of the `profile_t`'s friends
- Returns a `profile_t*`: a pointer to a newly created `profile_t` struct

Below is an example of `friendlen` and `init`:

```
1 char *get_example_addr(uint32_t id) { /* Omitted */ }
2
3 int main() {
4     uint32_t friends[] = {123456789, 1564, 0};
5     printf("%u", friendlen(friends));          // Prints 2
6     printf("%s", get_example_addr(987654321)); // Prints "10.97.192.161"
7
8     profile_t *new_profile = init(987654321, get_example_addr, friends);
9     printf("%u", new_profile->id);              // Prints 987654321
10    printf("%s", new_profile->addr);             // Prints "10.97.192.161"
11    printf("%u", new_profile->friends[0]);       // Prints 123456789
12    printf("%u", new_profile->friends[1]);       // Prints 1564
13    printf("%u", new_profile->friends[2]);       // Prints 0
14
15    return 0;
16 }
```

You may assume the following:

- Dynamic memory allocation will never fail.
- All relevant standard libraries have been imported.
- All arguments of the function are properly formatted and valid.
- All IDs are non-zero.

Useful function signatures:

```
1 void *malloc(size_t size);
2 void *calloc(size_t num_elements, size_t size);
3 void *realloc(void *ptr, size_t size);
4 void free(void *ptr);
5 size_t strlen(char *s);
6 char *strcpy(char *dest, char *src);
7 void *memcpy(void *dest, void *src, size_t size);
```

Implement `friendlen` to match the described behavior.

```
1 size_t friendlen(uint32_t *friends) {
2     size_t len = 0 ;
3     while (friends[len]) {
4         len++;
5     }
6     return len;
7 }
```

Q2.1
Q2.2
Q2.3
Q2.4

Implement `init` to match the described behavior. You may use `friendlen`, and you may assume that it matches the described behavior, but you may not assume any specific implementation.

```
1 profile_t *init(uint32_t id, char *(*addr_fn)(uint32_t), uint32_t *friends) {
2     profile_t *pf = calloc(1, sizeof *pf);
3     pf->addr = addr_fn(id);
4     pf->id = id;
5     size_t num_friends = friendlen(friends);
6     pf->friends = calloc(num_friends + 1, sizeof *pf->friends);
7     memcpy(pf->friends, friends, num_friends * sizeof(uint32_t));
8     return pf ;
9 }
```

Q2.5
Q2.6
Q2.7
Q2.8
Q2.9
Q2.10
Q2.11

For Q2.12 – Q2.13, which section of memory would the variable live in?

Q2.12 (1.5 points) `pf`

☐ Code ☐ Static ☒ Stack ☐ Heap

Q2.13 (1.5 points) `friends`

☐ Code ☐ Static ☒ Stack ☐ Heap

Q3 Floating Point

(13 points)

For Q3.1 – Q3.5, assume that we are using an IEEE-754 standard floating point representation with 20 bits: 1 sign bit, 6 exponent bits (with a standard bias of -31), and 13 significand bits.

Q3.1 (2 points) Convert 260 to hexadecimal.

Solution: $260 = 256 + 4 = 1.000001 \times 2^8$

After the bias, this means the exponent we need to encode is $8 + 31 = 39$ or **0b100111**.

Combining this with the sign and mantissa, **0b0 100111 0 0000 1000 0000**

0b0100 1110 0000 1000 0000

0x4E080

Grading: The sign, exponent, and mantissa bits were graded separately as all-or-nothing. A 1 point penalty was applied if the answer had incorrect number of bits.

Q3.2 (2 points) Convert 20.125×2^{-35} to hexadecimal.

Solution: $20.125 = 16 + 4 + 0.125 = 10100.001 \times 2^{-35}$

When we try to get this into normal form with an implicit 1, we get the following exponent:
 1.0100001×2^{-31}

This needs to be in denorm form, so we get the following if we move the decimal point over 1 more time: $0.10100001 \times 2^{-31}$

Sign: **0b0**

Exponent: **0b00 0000**

Mantissa: **0b1 0100 0010 0000**

Combining all these together, we get:

0b0000 0001 0100 0010 0000

0x01420

Q3.3 (2 points) Convert **0xC6880** to decimal.

Solution: **0xC6880 = 0b1100 0110 1000 1000 0000**

Sign: **0b1**

Exponent: **0b10 0011 = 35 - 31 = 4**

Mantissa: **0b0 1000 1000 0000**

Combining all the elements together, we get the following:

$-1.0100010000000 \times 2^4$

$-10100.010000000 \times 2^0$

-20.25

Q3.4 (2 points) What is the largest representable non-infinite number, in hexadecimal?

Solution: Sign: We want our sign to be positive, so our sign is **0b0**

Exponent: A non-infinite number means our exponent field cannot be all 1's. As a result, our exponent will be **0b11 1110**

Mantissa: If we want to create the largest representable non-infinite number, we want our mantissa to be all 1's **0b1 1111 1111 1111**

Combining all of this together, we get: **0111 1101 1111 1111 1111**

0x7DFFF

Q3.5 (3 points) Assuming $+0$ and -0 are two unique values, how many unique, representable, non-infinite, non-NaN values are there? You may represent your answer in terms of powers of two.

Solution: In total, we have 20 bits in our floating point system. We can represent 2^{20} unique values with 20 bits. We then have to subtract infinity, and NaN values. Both infinity and NaN's happen when our exponent is all 1. We have 13 bits in our mantissa, so we subtract 2^{13} . However, we need to account for the sign bit, so in total, we have

$$2^{20} - 2 \times 2^{13}$$

$$2^{20} - 2^{14}$$

Q3.6 (2 points) If we have an IEEE-754 standard floating point representation with 1 sign bit and 6 exponent bits (with a standard bias of -31), what is the minimum number of significand bits needed to represent 2^{-47} ?

Solution: Using this floating point scheme, the lowest our exponent can be is 2^{-30} . To fully represent 2^{-47} , the minimum number of significand bits we need is 17.

Q4 61Caterpillars

(22 points)

Implement `is_equal`, a RISC-V function, as follows:

- Input `a0`: a 32 bit value
- Input `a1`: a 32 bit value
- Returns in `a0`: 1 if `a0 == a1`, otherwise 0

You may not use `beq` or `bne` in your implementation of `is_equal`.

```
1 is_equal:
2     blt a0 a1 return_false
3     blt a1 a0 return_false
4 return_true:
5     addi a0 x0 1
6     jr ra
7 return_false:
8     addi a0 x0 0
9     jr ra
```

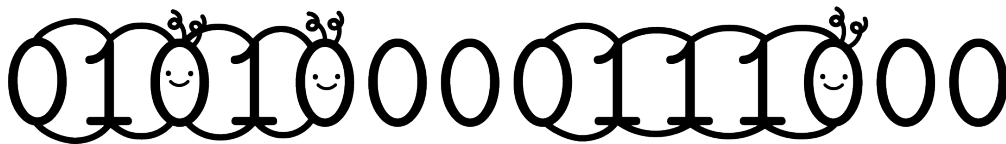
A 61Caterpillar is a sequence of bits, beginning with a 0, followed by one or more 1s, and ending with a 0. For example, `0b011110`, `0b010`, and `0b011111110` are 61Caterpillars, while `0b00`, `0b110`, `0b10001`, `0b1111`, and `0b01111` are not.

A zero can be shared by two 61Caterpillars. For example, `0b01010` is two caterpillars.

Implement `count_caterpillars`, a RISC-V function, as follows:

- Input `a0`: a 32 bit value, **where the most significant bit is 0**
- Returns in `a0`: the number of 61Caterpillars `a0` contains

For example, calling `count_caterpillars` where `a0 = 0x00001438 = 0b0...01 0100 0011 1000` should return 3.



You may use `is_equal`, and you may assume that it matches the described behavior, but you may not assume any specific implementation. `count_caterpillars` must follow calling convention and must be memory efficient.

```
1 count_caterpillars:
2     addi sp sp -16
3         Q4.3
4     sw ra 0(sp)
5         Q4.4
6     sw s4 4(sp)
7         Q4.5
8     addi s4 x0 0 # number of caterpillars
9     addi a1 x0 2
10         Q4.6
11 loop:
12     sw a0 8(sp)
13         Q4.7
14     sw a1 12(sp)
15         Q4.8
16     andi a0 a0 3
17         Q4.9
18     jal ra is_equal
19         Q4.10
20     add s4 s4 a0
21         Q4.11
22     lw a0 8(sp)
23         Q4.12
24     lw a1 12(sp)
25         Q4.13
26     srli a0 a0 1
27         Q4.14
28     bne a0 x0 loop
29         Q4.15
30     mv a0 s4
31     lw ra 0(sp)
32         Q4.16
33     lw s4 4(sp)
34         Q4.17
35     addi sp sp 16
36         Q4.18
37     jr ra
```

Q5 Boolean Algebra**(9 points)**

For Q5.1 – Q5.3, select the Boolean operator that matches the behavior of the truth table.

Q5.1 (1 point)

X	Y	Out
0	0	0
0	1	0
1	0	0
1	1	1

- ☒ AND
- ☐ NAND
- ☐ OR
- ☐ NOR
- ☐ None of the above

Q5.2 (1 point)

Y	Z	Out
0	0	1
0	1	0
1	0	0
1	1	1

- ☐ AND
- ☐ NAND
- ☐ OR
- ☐ NOR
- ☒ None of the above

Q5.3 (1 point)

X	Y	Out
0	0	0
0	1	1
1	0	1
1	1	1

- ☐ AND
- ☐ NAND
- ☒ OR
- ☐ NOR
- ☐ None of the above

For Q5.4 – Q5.7, simplify the given Boolean expression. You must use at most the number of operators given. NOT (!), AND (*), OR (+) each count as one operator. We will assume standard C operator precedence, so use parentheses when uncertain.

Your answer may consist of the following characters:

Inputs	NOT	AND	OR	Constants	Parentheses
A, B, C	!	*	+	0, 1	()

Q5.4 (1.5 points) $(A * B) + (A * !B) + (!A * !B)$ (2 operators)

Solution: $A + !B$

$(A * B) + (A * !B) + (!A * !B)$

The first thing we do is use the distributive property to get:

$(A * B) + !B * (A + !A)$

$(A * B) + !B$

We can add back the $(A * !B)$ to this result to get:

$(A * B) + (A * !B) + !B$

We can then use the distributive property one more time to get our final solution:

$A * (B + !B) + !B$

$A + !B$

Q5.5 (1.5 points) $C * (A + B) + !A * (!B + C)$ (3 operators)

Solution: $C + !(A + B)$

$C * (A + B) + !A * (!B + C)$

We can use the distributive property to get:

$A * C + B * C + !A * !B + !A * C$

There is a common C term that we can distribute to get:

$C * (A + !A + B) + !A * !B$

$C + !A * !B$

We can use DeMorgan's Law to get:

$C + !(A + B)$

Q5.6 (1.5 points) $(A + B) * (A * C + B * (A + C))$ (4 operators)

Solution: The following 3 solutions got full credit: $A * (C + B) + B * C$, $B * (A + C) + A * C$, $C * (A + B) + A * B$

$(A + B) * (A * C + B * (A + C))$

We can use the distributive property to get:

$(A + B) * (A * C + A * B + B * C)$

We can use the distributive property again to get:

$A * C + A * B + A * B * C + A * B * C + A * B * C + A * B + B * C$

And then combine common terms:

$A * C + A * B * C + A * B + B * C$

We can choose one of the following terms to use the distributive property: $A * C$, $A * B$, $B * C$. The solution will use the $A * C$ term.

$A * C * (1 + B) + A * B + B * C$

$A * C + A * B + B * C$

We can use the distributive property again to get:

$A * (C + B) + B * C$

Q5.7 (1.5 points) $!(C + B) * (A * !C * (!A + B))$ (0 operators)

Solution: 0

$!(C + B) * (A * !C * (!A + B))$

Apply DeMorgan's on the left term to get:

$(!C * !B) * (A * !C * (!A + B))$

Use the distributive property on the right term:

$(!C * !B) * (A * (!C * !A + !C * B))$

Use the distributive property on the right term again to get:

$(!C * !B) * (A * !C * !A + A * !C * B))$

$(!C * !B) * (A * !A * !C + A * !C * B))$

$(!C * !B) * (0 + A * !C * B))$

$(!C * !B) * (A * !C * B))$

We then apply the distributive property again:

$(!C * !B * A) * (!C * !B * !C) * (!C * !B * B)$

The last term becomes a 0, so we get:

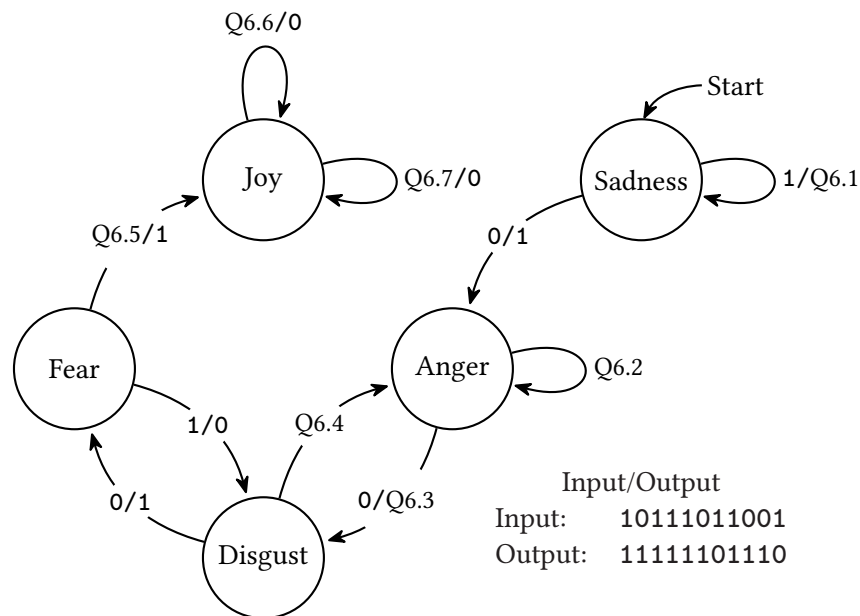
$(!C * !B * A) * (!C * !B * !C) * 0$

0

Q6 *fINite State machIne DEbacle, OUTsmarted*

(7 points)

Uh oh! Riley's emotions are scrambled, and she's forgotten how to go from one emotion (state) to the next. Select the state transitions such that the FSM matches the input/output provided.



Q6.1 (1 point)

☐ 0

☒ 1

Q6.2 (1 point)

☐ 0/0

☐ 0/1

☐ 1/0

☒ 1/1

Q6.3 (1 point)

☐ 0

☒ 1

Q6.4 (1 point)

☐ 0/0

☐ 0/1

☒ 1/0

☐ 1/1

Q6.5 (1 point)

☒ 0

☐ 1

Q6.6 (1 point)

☒ 0

☐ 1

Q6.7 (1 point)

☐ 0

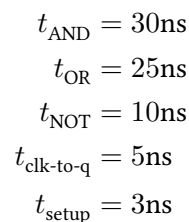
☒ 1

(Question 6 continued...)

Solution: Grading: All-or-nothing, except for the last two questions, where partial was awarded for having one correct answer between the two. The answers to Q6.6 and Q.7 are interchangeable.

(8 points)

For the entirety of this question, consider the following circuit and timings. Assume all registers are connected to the same clock and **A_in**, **B_in**, and **C_in** will not cause any hold time violations.



Q7.1 (2 points) What is the **longest combinational logic delay**, in nanoseconds?

Solution: 80ns. The longest path goes through the top OR gate, the AND gate, and the other OR gate.

Q7.2 (2 points) What is the **shortest combinational logic delay**, in nanoseconds?

Solution: 30ns. The shortest path is from the output of the rightmost register, through the NOT gate, and the rightmost OR gate.

Q7.3 (2 points) What is the **minimum allowable clock period** for this circuit to function properly, in nanoseconds?

Solution: 88ns. Take your answer from 7.1 and add 8; 3ns for the clk-to-q and 5ns for the setup.

Q7.4 (2 points) What is the **maximum hold time** the registers can have such that there are no hold time violations, in nanoseconds?

Solution: 35ns. Take your answer from 7.2 and add 5ns for the clk-to-q.

Q8 *The Finish Line*

(0 points)

These questions will not be assigned credit; feel free to leave them blank.

Q8.1 (0 points)

☐ Apple Juice

☐ Orange Juice

Q8.2 (0 points)

☐ Cat

☐ Dog

Q8.3 (0 points) If there's anything else you want us to know, or you feel like there was an ambiguity in the exam, please put it in the box below.

For ambiguities, you must qualify your answer and provide an answer for both interpretations. For example, "if the question is asking about A, then my answer is X, but if the question is asking about B, then my answer is Y". You will only receive credit if it is a genuine ambiguity and both of your answers are correct. We will only look at ambiguities if you request a regrade.

Solution: $\neg_(\neg)_/\neg$